



Values of floating-point types

Document number:

[P3938R1](#)

Date:

2026-02-20

Audience:

SG6, EWG, CWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3938/github

Source:

github.com/eisenwave/cpp-proposals/blob/master/src/floating-point-values.cow



ABSTRACT: It is not specified what values a floating-point type may represent in C++, leading to an unclear model for floating-point types. This paper introduces the necessary wording to specify what may be found between the lines of the standard.

§ Contents

- 1 **Revision history**
 - 1.1 Changes since R0
- 2 **Introduction**
- 3 **Q&A**
 - 3.1 Do infinity and NaN exist from a core language perspective?
Can there be an unsigned zero, infinity, or NaN value, or are all floating-point values signed?
 - 3.2 Conversely, can there be a signed zero, infinity, or NaN value?
Can there be a negative zero and an unsigned infinity, or is the "signedness requirement" all-or-none?
 - 3.3 Is negative zero negative? What about infinity and NaN?
 - 3.4 Is "negative NaN" distinct from "positive NaN"?
 - 3.5 Are different NaN payloads distinct values?
 - 3.6
 - 3.7



3.8	Can an extended floating-point type have no finite values?
3.9	Does the core language need to distinguish between normal/subnormal numbers?
3.10	Are there values beyond finite values, infinity, and NaN?
3.11	What does it mean for a type to “adhere to ISO/IEC 60559”?
3.12	Is <code>0.0</code> positive or negative zero?
3.13	Are arithmetic operations required to preserve the sign of zero?
3.14	How does template argument equivalence work for floating-point types?
4	Impact on the standard
5	Impact on implementations
6	Wording
6.1	[lex]
6.2	[basic]
6.3	[expr]
6.4	[temp]
6.5	[support]
6.6	[meta]
7	References

§ 1. Revision history

§ 1.1. Changes since R0

- Rationalized the status quo in §3.5. [Is negative zero negative? What about infinity and NaN?](#)
- Rebased §6. [Wording](#) on N5032
- In §6. [Wording](#), fixed improper handling of negative zero *floating-point-literals*
- In §6. [Wording](#), referenced [\[CWG3129\]](#)
- In §6. [Wording](#), improved handling of signaling NaNs by relaxing the definition of “adheres to ISO/IEC 60559”

§ 2. Introduction

The core language wording in the C++ standard does not specify what values a floating-point type may represent. There are a few questions that have no obvious answer:

- Do infinity and NaN exist from a core language perspective?



- Can there be an unsigned zero, infinity, or NaN value, or are all floating-point values signed?
- Conversely, can there be a signed zero, infinity, or NaN value?
- Can there be a negative zero and an unsigned infinity, or is the "signedness requirement" all-or-none?
- Are negative zero and positive zero distinct values? That is, do they compare equal, and if so, is there an observable difference (beyond looking at the sign bit) between them?
- Is negative zero negative? That is, when a *Preconditions* element requires a "non-negative" value, is the behavior undefined when negative zero is provided?
- Similarly, are negative infinity and negative NaN negative?
- Is "negative NaN" distinct from "positive NaN", or are these effectively the same NaN values with different "payloads"?
- Are different NaN payloads distinct values? If so, does that imply `std::has_unique_object_representations_v<std::float32_t>` is `true`?
- Can an extended floating-point type be so imprecise that it is incapable of representing any number? That is, could an *infinity-t* type in the style of `nullptr_t` be considered a floating-point type?
- Does the core language need to distinguish between normal and subnormal numbers?
- Are there any other possible categories of values beyond finite values, infinity, and NaN?
- What does it mean for a type to “adhere to ISO/IEC 60559”, as mentioned in `std::numeric_limits::is_iec559`?
- Is `0.0` positive or negative zero, or is it implementation-defined/unspecified?
- Are arithmetic operations required to preserve the sign of zero?
- How does template argument equivalence work for floating-point types?

Bits of information may be found in various parts of the standard, such as in the concept of "adhering to ISO/IEC 60559", `numeric_limits` requirements, the inheritance of C features such as `std::fpclassify`, etc. However, some of these questions are so deeply unclear that a core issue alone wouldn't be sufficient to solve the problem.

The goal of this paper is to answer these questions, not by making any evolutionary changes to the language, but by investigating what the status quo is and turning that into wording.

§ 3. Q&A

In the following subsections, the paper tries to find a good answer to the questions above. These answers are primarily based on the existing wording and on existing implementation practice.

§ 3.1. Do infinity and NaN exist from a core language perspective?

Yes. [\[basic.fundamental\]](#) mentions infinity. While NaN is not mentioned explicitly, `numeric_limits::quiet_NaN()` implies it.

At the very least, floating-point types may represent



- zero, for zero-initialization to make sense,
- “negative zero”, since both C and C++ already mention this term many times, and
- infinity, qNaN, and sNaN, [\[numeric.limits\]](#) to make sense and to have types that adhere to ISO/IEC 60559.

§ 3.2. Can there be an unsigned zero, infinity, or NaN value, or are all floating-point values signed?

There can be fully unsigned floating-point numbers. The C standard explicitly mentions unsigned infinity and unsigned zero, and different NaN signs are typically not distinct values. The C++ standard presumably inherits the C model because all of the `<cmath>` functions are stated to work like C23's `<math.h>`. There is no C++ restriction on `fpclassify`.

Furthermore, [\[numeric.limits.members\]](#) states that `numeric_limits::is_signed` is meaningful for all specializations, so there is presumably no requirement that a floating-point type or any of its values are signed.

§ 3.3. Conversely, can there be a signed zero, infinity, or NaN value?

Presumably. The C++ standard mentions negative zero and negative infinity in a number of places. NaN with negative sign bit and NaN with positive sign bit are usually not considered distinct values (e.g. ISO/IEC 60559), but the C++ standard has no wording that would prohibit that distinction.

§ 3.4. Can there be a negative zero and an unsigned infinity, or is the "signedness requirement" all-or-none?

It's not all-or-none. There are floating-point types that adhere to ISO/IEC 60559, and these do not have a distinct negative and positive NaN, so it cannot be all-or-none.

§ 3.5. Is negative zero negative? What about infinity and NaN?

No. The wording uses “negative” in the “less than zero” sense ([\[complex.numbers\]](#) `polar`, `complex`). The C23 standard defines a “negative value” to values which are less than zero, so negative zero and “negative NaN” (NaN with negative sign bit) are not negative.

Negative infinity is negative because it compares less than zero.



NOTE: Of course, we could always change our use of the term so that negative zero is negative. This may make it easier to word the handling of negative and positive numbers in division by zero, if that was ever made well-defined. For example, `-0.0 / numeric_limits::infinity()` should produce negative infinity, if well-defined, which it currently isn't.

However, this would be a major deviation from the C wording, and we usually try to keep the C++ floating-point wording symmetrical. Also, the ISO/IEC 60559 phrase “has negative sign bit” includes negative zero as well. In most contexts, negative zero is treated the same as positive zero, and it would impose a wording burden if one was negative but not the other.

§ 3.6. Is “negative NaN” distinct from “positive NaN”?

Presumably. While ISO/IEC 60559 does not consider NaNs with different sign bit to be distinct values, the C++ standard does not prohibit such a model.

§ 3.7. Are different NaN payloads distinct values?

Sometimes. `numeric_limits::quiet_NaN()` and `numeric_limits::signaling_NaN()` are two NaNs distinguished only by payload. However, not every implementation treats these distinctly. For example, when compiling to WASM, none of the f32 or f64 instructions handle signaling NaNs.

§ 3.8. Can an extended floating-point type have no finite values?

No. This would make zero-initialization unimplementable, and would make `numeric_limits` members such as `epsilon()` worded in a nonsensical way.

§ 3.9. Does the core language need to distinguish between normal/subnormal numbers?

No. Subnormals either get “flushed” to zero, meaning that they are alternative representations of zero, numerically, or they are simply finite numbers like any other. The classification of “normal” and “subnormal” is an implementation detail of the floating-point format.

§ 3.10. Are there values beyond finite values, infinity, and NaN?

The exotic and surprising part is that floating-point types may represent additional implementation-defined values beyond finite values, infinity, and NaN. This is backed by

- C23 §5.2.5.3.3 [Characteristics of floating types <float.h>] paragraph 8, and
- C23 §7.12 [Mathematics <math.h>] paragraph 12.

These paragraphs support the existence of such additional implementation-defined classifications. C++ inherits the behavior of `fpclassify` from `<math.h>`, so it must support the same classifications to be compatible.



EXAMPLE: [NetBSD-`fpclassify`] documents that the `fpclassify` macro for VAX floating-point numbers may yield `FP_ROP` (reserved operand), which raises a reserved operand fault, i.e. a CPU exception when processed, similar to integer division by zero.

This is different from signaling NaN in that by default, signaling NaN only raises a floating-point exception but doesn't trap.



While neither the C23 wording nor the C++ wording handles these extra implementation-defined classifications well, they nonetheless exist, and it seems like an unmotivated breaking change to drop support for them.

§ 3.11. What does it mean for a type to “adhere to ISO/IEC 60559”?

It just means that the value representation is one of binary16, binary32, binary64, binary128, or some extended or decimal floating-point format specified in ISO/IEC 60559.

When it comes to operations, the C++ standard specifies no mapping between expressions (e.g. `/`) and the ISO/IEC 60559 operations (e.g. “division”). In fact, it seems to deliberately deviate from the ISO/IEC 60559 operations by declaring division by zero to be undefined behavior, even for floating-point types ([\[expr.mul\]](#)).



EXAMPLE: This implies that e.g. `numeric_limits<float>::infinity() + 0` is not infinity, but “UB by omission” or “UB by wording hole”, even if `float` “adheres to ISO/IEC 60559”.

While it would be desirable to align the C++ operations with ISO/IEC 60559 operations, this would require significant wording effort. There exists no core wording that even attempts at doing so. Note that implementations don't always compute results correctly rounded (with the greatest available precision), while the ISO/IEC 60559 operations are usually correctly rounded. Therefore, aligning the C++ expressions with ISO/IEC 60559 may require changes to most implementations.

§ 3.12. Is `0.0` positive or negative zero?

Presumably positive. The C++ standard does not mandate a specific sign for *floating-point literals*, but no implementation makes them negative, nor does it make practical sense to do so. We should mandate that literals have no negative sign.

§ 3.13. Are arithmetic operations required to preserve the sign of zero?

Presumably no. While the ISO/IEC 60559 multiplication operation preserves the sign bit when multiplying with 1, the C++ standard does not clearly mandate such behavior.

Especially considering that not every type adheres to ISO/IEC 60559, we should require a specific handling of sign bits, explicitly. For example, the unary `-` operator should be required to flip the sign bit even for zero, otherwise `-0.0` may not be a spelling of negative zero, as users rely on.

§ 3.14. How does template argument equivalence work for floating-point types?



In practice, it is based on bitwise identical values. [\[temp.type\]](#) states that

”

Two values are *template-argument-equivalent* if they are of the same type and

- they are of integral type and their values are the same, or
- they are of floating-point type and their values are identical, or
- [...]

The distinction between “same” and “identical” is not obvious. It was added during C++20 NB comment resolution by [\[P1907R1\]](#), after [\[P1714R1\]](#) (the paper which originally added support for floating-point template parameters) was rejected. The original paper used the terminology “identical value representations”, which makes the intent obvious, unlike the current wording.

GCC, Clang, and MSVC implement the design of the original paper by mangling the bit-casting to an integer and mangling it into the name.



EXAMPLE: Instantiating `f` with two different qNaNs payloads results in two distinct instantiations, despite there only being only one distinct qNaN value, at least from an ISO/IEC 60559 perspective.

```
template <float>
void f() {}

// "default" qNaN:
template void f<std::bit_cast<float>(0x7fc00000)>>();
// also qNaN (same value, different bit pattern):
template void f<std::bit_cast<float>(0x7fc00001)>>();
```

Both GCC and Clang emit the assembly:

```
_Z1fILf7fc00000EEvv:
    ret
_Z1fILf7fc00001EEvv:
    ret
```

The wording should be clarified to match the design of the original paper and to match what implementations actually do. Creating a distinct instantiation for every bit pattern is the most useful behavior anyway; it would be impossible to deliberately wrap a NaN payload of choice in `std::constant_wrapper` otherwise, for example.

§ 4. Impact on the standard

To clarify the floating-point specification, a bit of additional wording is required.

This paper only “picks the low-hanging fruits”, so to speak. In the long run, specifying the handling of NaNs and infinities by C++ expressions, documenting ISO/IEC 60559 conformance, and other large changes may be desirable. However, those would require much greater changes.

§ 5. Impact on implementations

All proposed wording changes document the current behavior of major implementations.

§ 6. Wording

The changes are relative to [\[N5032\]](#).

§ [lex]

Change [\[lex.fcon\]](#) paragraph 3 as follows:



If the scaled value is not in the range of representable values for its type, the program is ill-formed. Otherwise, the value of a *floating-point-literal* is the scaled value if representable, else the larger or smaller representable value nearest the scaled value, chosen in an implementation-defined manner. The value of a *floating-point-literal* never has negative sign bit.

If [\[CWG3129\]](#) has not yet been approved at the time of writing, add a new example attached to [\[lex.fcon\]](#) paragraph 3:



[Example: `0.0` is positive zero, and `-0.0` is negative zero if `double` has a signed zero ([\[basic.fundamental\]](#), [\[expr.unary.op\]](#)). — end example]

Otherwise, if the proposed resolution of [\[CWG3129\]](#) has been merged, change the existing example as follows:



[Example: The following example assumes that `std::float32_t` is supported ([\[basic.extended.fp\]](#)).

```
std::float32_t x = 0.0f32;           // value 0 positive zero is exactly representable
std::float32_t x = -0.0f32;          // negation of positive zero equals negative zero

std::float32_t y = 0.1f32;           // rounded to one of two values nearest to 0.1
std::float32_t z = 1e1000000000f32; // either greatest finite value or positive infinity

— end example]
```

§ [basic]

Immediately preceding [\[basic.fundamental\] paragraph 13](#), insert three new paragraphs:



A floating-point type *adheres to ISO/IEC 60559* if its value representation is one of the floating-point formats specified in ISO/IEC 60559 and can represent the sets of floating-point values listed in ISO/IEC 60559, except that it is implementation-defined whether all signaling NaN values are treated as a quiet NaN instead.

[Note: Adherence to ISO/IEC 60559 does not imply that operations on floating-point types behave exactly as specified in that standard. For example, the behavior of addition in C++ ([\[expr.add\]](#)) can differ from the “addition” operation in ISO/IEC 60559. — *end note*]



DRAFTING NOTE: See § [\[support\]](#) for why signaling NaN is being treated specially.



DRAFTING NOTE: The following change resolves [CWG submission #819](#).



A floating-point type shall at least represent a subset of rational numbers. Depending on the implementation-defined value representation for the type, it may additionally represent the non-finite values

- infinity,
- quiet “Not a Number” value,
- signaling “Not a Number” value, and
- further implementation-defined values.

For any of the above (including zero), a floating-point type may represent either a single unsigned value or two distinct values with negative and positive sign.

[Note: A floating-point type which adheres to ISO/IEC 60559 is capable of representing a negative and positive variant of finite values and infinity, an unsigned quiet NaN, and an unsigned signaling NaN. Such a type has multiple object representations for quiet NaN and signaling NaN. — *end note*]



DRAFTING NOTE: The following change resolves [CWG submission #817](#).

Immediately preceding [\[basic.fundamental\] paragraph 14](#), insert a new paragraph:



A value is *negative* if and only if it compares less than 0 ([\[expr.rel\]](#)).

[Note: Thus, negative zeros and NaNs are not negative values. — *end note*]

§ [\[expr\]](#)

At the end of [\[expr.pre\]](#), insert a new paragraph:



Unless otherwise stated, it is unspecified which of the alternative representations for a value is chosen as the result of an expression. Furthermore, if the result is of a floating-point type that can represent negative and positive zero, it is implementation-defined which zero is chosen as the result of the expression.

Change [\[conv.fpint\]](#) paragraph 2 as follows:



A prvalue of an integer type or of an unscoped enumeration type can be converted to a prvalue of a floating-point type. The result is exact if possible. If the value being converted is zero, the result is positive or unsigned zero. If the value being converted is in the range of values that can be represented but the value cannot be represented exactly, it is an implementation-defined choice of either the next lower or higher representable value.

[Note: Loss of precision occurs if the integral value cannot be represented exactly as a value of the floating-point type. — end note]

If the value being converted is outside the range of values that can be represented, the behavior is undefined. If the source type is `bool`, the value `false` is converted to zero and the value `true` is converted to one.

Change [\[expr.unary.op\]](#) paragraph 8 as follows:



The operand of the unary `-` operator shall be a prvalue of arithmetic or unscoped enumeration type and the result is the negative of its operand. Integral promotion is performed on integral or enumeration operands. The negative of an unsigned quantity is computed by subtracting its value from 2^n , where n is the number of bits in the promoted operand. For floating-point types that may represent negative and positive zero, the unary `-` operator results in the zero with opposite sign. The type of the result is the type of the promoted operand.



DRAFTING NOTE: The following changes resolve [CWG submission #814](#).

Change [\[expr.spaceship\]](#) paragraph 4 as follows:



If both operands have arithmetic types, or one operand has integral type and the other operand has unscoped enumeration type, the usual arithmetic conversions are applied to the operands. Then:

- If a narrowing conversion ([\[cl.init.list\]](#)) is required, other than from an integral type to a floating-point type, the program is ill-formed.
- Otherwise, if the operands have integral type, [...].
- Otherwise, the operands have floating-point type, and the result is of type `std::partial_ordering`. The expression `a <=> b` yields `std::partial_ordering::less` if `a` is less than `b`, `std::partial_ordering::greater` if `a` is greater than `b`, `std::partial_ordering::equivalent` if `a` is equivalent to `b`, and

`std::partial_ordering::unordered` otherwise. Positive zeros are equivalent to negative zeros.



Change [\[expr.rel\]](#) paragraph 6 as follows:



If both operands (after conversions) are of arithmetic or enumeration type, each of the operators shall yield `true` if the specified relationship is `true` and `false` if it is `false`. Positive zeros compare equal to negative zeros.

Change [\[expr.eq\]](#) paragraph 8 as follows:



If both operands are of arithmetic or enumeration type, the usual arithmetic conversions ([\[expr.arith.conv\]](#)) are performed on both operands; each of the operators shall yield `true` if the specified relationship is `true` and `false` if it is `false`. Positive zeros compare equal to negative zeros.

§ **[temp]**



DRAFTING NOTE: The following changes resolve [Editorial PR #8375](#).

Immediately preceding [\[temp.type\]](#) paragraph 2, insert a new paragraph:



Two values x and y of a type T are *bitwise identical* if `std::bit_cast<U>(x) == std::bit_cast<U>(y)` ([\[bit.cast\]](#)) is `true`, where U is a hypothetical unsigned integer type with the same size as T and the same amount and positioning of padding bits in its object representation as T .

[Note: It is possible that two values are the same but not bitwise identical. For example, a floating-point type T can have multiple representations of zero, of any other finite number, of the same NaN value, and more, even if the object representation of T has no padding bits. — end note]



DRAFTING NOTE: The ISO/IEC 60559 standard also uses the term “bitwise identical”.

Change [\[temp.type\]](#) paragraph 2 as follows:



Two values are *template-argument-equivalent* if they are of the same type and

- they are of integral type and their values are the same, or
- they are of floating-point type and their values are bitwise identical, or
- they are of type `std::nullptr_t`, or
- [...]



DRAFTING NOTE: We cannot simply say that the value representations of the template arguments is equal because we are talking about values, not objects, so a value and object representation isn't available in the first place.

While it would be possible to define “bitwise identical” without the use of `std::bit_cast`, the use of `std::bit_cast` makes the intent obvious at first glance. The use of “unsigned” may seem unnecessary, but it makes the wording more obvious, and there exist integer types such as `bool` which don't behave like the hypothetical type we want here.



DRAFTING NOTE: This wording approach implies that template arguments have a value representation that can be used to distinguish different template specializations all using the same qNaN argument with different payloads. The rest of the wording may not actually support this, but that is how every implementation works.

§ [support]

Change `[numeric.limits.members]` as follows:



```
static constexpr bool has_signaling_NaN;
```

`true` if the type has a representation for a signaling “Not a Number”.

Meaningful for all floating-point types.

Shall be `true` for all specializations in which `is_iec559 != false`.

[Note: An implementation is permitted to treat all signaling NaNs as quiet NaNs, even for types that adhere to ISO/IEC 60559 ([basic.fundamental]). In that case, `has_signaling_NaN` can be true despite quiet and signaling NaNs being functionally equivalent. — end note]



DRAFTING NOTE: While ISO/IEC 60559 requires a signaling NaN to exist for its interchange formats (see ISO/IEC 60559 §6.2 Operations with NaNs), this is ignored by implementations in practice. For example, GCC by default and Clang by default for WASM targets report `is_iec559` for `float`, despite there not existing a signaling NaN. Paradoxically, even when there is no signaling NaN, GCC and Clang accept the following:

```
#include <limits>
static_assert(std::numeric_limits<float>::is_iec559); // OK
static_assert(std::numeric_limits<float>::has_signaling_NaN); // OK even
```

The worst possible outcome would be to force implementations to report `is_iec559` as `false` on WASM. However, since the requirement for “adhering to ISO/IEC 60559” in

§ [basic] has been relaxed, the implementation is allowed to treat signaling NaNs as quiet NaNs, which fixes this contradiction.



Change [cmp.alg] paragraph 2 as follows:



The name `weak_order` denotes a customization point object ([`customization.point.object`]). Given subexpressions `E` and `F`, the expression `weak_order(E, F)` is expression-equivalent ([`defns.expression.equivalent`]) to the following:

- [...]
- Otherwise, if the decayed type `T` of `E` is a floating-point type, yields a value of type `weak_ordering` that is consistent with the ordering observed by `T`'s comparison operators and `strong_order`, and if `numeric_limits<T>::is_iec559` is `true`, is additionally consistent with the following equivalence classes, ordered from lesser to greater:
 - together, all **negative** NaN values with negative sign bit;
 - negative infinity;
 - each normal negative value;
 - each subnormal negative value;
 - together, both zero values;
 - each subnormal positive value;
 - each normal positive value;
 - positive infinity;
 - together, all **positive** NaN values with positive sign bit.
- [...]



DRAFTING NOTE: ISO/IEC 60559 floating-point types have no “negative NaN” or “positive NaN”, so the existing wording makes no sense, especially not with the new definition of “negative”.

§ [meta]

Change [meta.unary.prop] paragraph 10 as follows:



The predicate condition for a template specialization `has_unique_object_representations<T>` shall be satisfied if and only if

- `T` is trivially copyable, and
- any two objects of type `T` with the same value have the same object representation, where

- two objects of array or non-union class type [...]
- two objects of union type [...]



The set of scalar types ([basic.fundamental]) for which this condition holds is implementation-defined.

[Example: The condition does not hold for floating-point types that adhere to ISO/IEC 60559 because there are multiple representations of the same NaN value. The condition also does not hold for any type whose object representation contains padding bits. — end example]

§ 7. References

[N5032] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-12-15)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf>

[P1714R1] *Jorg Brown*. NTTP are incomplete without float, double, and long double! (Revision 1) (2019-07-19)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1714r1.html>

[P1907R1] *Jens Maurer*. Inconsistencies with non-type template parameters (2019-11-08)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1907r1.html>

[CWG3129] *Jan Schultke*. Clarify which floating-point-literals are valid (2025-11-10)

https://www.open-std.org/jtc1/sc22/wg21/docs/cwg_active.html#3129

[NetBSD-fpclassify] *fpclassify(3)* - NetBSD Manual Pages

<https://man.netbsd.org/NetBSD-10.1/fpclassify.3>